

260602 Franka LLM Drawing Project 현재 진행 상황 정리

작성일: 2026-06-02 목적: 팀원에게 현재 코드 구조, Isaac Sim 실행 흐름, 발표용 데모 상태, 앞으로의 확장 방향을 공유하기 위한 문서

1. 현재 프로젝트 한 줄 요약

현재 프로젝트는 **자연어처럼 입력한 drawing command를 DrawingPlan JSON으로 바꾸고, 그 JSON을 JADE 기반 로봇 실행 계층에 통과시켜 Isaac Sim에서 Franka가 펜으로 도형을 따라 움직이게 하는 구조**까지 구현되어 있다.

아직 완성된 최종 시스템은 아니다. 현재는 OpenAI API 비용 없이 재현 가능한 no-api fallback을 사용해 발표용 end-to-end 데모를 안정적으로 보여주는 단계이다.

자연어 입력

- > no-api fallback planner
- > DrawingPlan JSON
- > bridge validator
- > JADE trajectory/control
- > Isaac Sim Franka 실행
- > 로그/그래프 저장

2. 현재 폴더별 역할

Unit_Action_Langchain-main/

기존 팀 프로젝트의 LLM planner 계층이다.

원래 목적은 자연어 명령을 DrawingPlan JSON으로 바꾸는 것이다.

IK, Jacobian, joint command, Isaac 실행은 담당하지 않는다.

llm_isaac_bridge/

이번에 추가한 연결부이다.

기존 LLM planner와 현재 Isaac 제어 코드를 직접 섞지 않고,

자연어 입력, plan 생성, validation, Isaac 실행 명령 생성을 담당한다.

현재 발표 데모에서는 OpenAI API 없이 no-api fallback을 사용한다.

franka_llm_drawing/

현재 실제로 사용 중인 로봇 제어 core package이다.

DrawingPlan JSON을 trajectory로 바꾸고, frame transform, pen-tip offset,

DLS IK, normal-force admittance, JADE 평가, Isaac backend 실행을 담당한다.

configs/

frame, Isaac scene, JADE controller, sampling parameter 설정 파일이 들어 있다.

usd/

Franka, table, paper, pen, light가 포함된 Isaac USD scene이 들어 있다.

examples/

Isaac 실행 스크립트, offline sampling, sample plan 등이 들어 있다.

outputs/

실행 결과, validation report, plot, CSV log가 저장된다.

plan_and_information/

프로젝트 방향, 제어 전략, 좌표계 검수, 데모 계획 등 설명 문서가 들어 있다.

3. 현재 구현된 핵심 기능

3.1 USD / Isaac 환경

- usd/franka_drawing_scene_clean.usda 기준 scene을 사용한다.
- Franka는 외부 URL이 아니라 프로젝트 내부 asset에서 참조한다.
- table, paper, pen, light가 구성되어 있다.

- paper surface는 /World/BoardFrame과 맞춰져 있다.
- 펜은 Franka 말단부에 부착되어 있고, 실제 제어 target은 PenTipFrame이다.
- 조명은 robot, table, paper, pen 움직임이 잘 보이도록 추가되어 있다.

3.2 제어 대상

Isaac backend는 반드시 Franka arm joint 7개만 command 대상으로 사용한다.

```
panda_joint1
panda_joint2
panda_joint3
panda_joint4
panda_joint5
panda_joint6
panda_joint7
```

finger joint는 USD articulation 안에 남아 있을 수 있지만, 제어기 command 대상에는 포함하지 않는다.

3.3 DrawingPlan 처리

LLM 또는 fallback planner가 만든 DrawingPlan JSON은 다음 unit action들을 포함한다.

```
move_to_start
align_pen_orientation
pen_down
draw_line
draw_arc
pen_up
```

이 action들은 바로 joint command가 아니다. 먼저 trajectory planner를 거쳐 board-frame pen-tip pose sample로 바뀐다.

3.4 Trajectory planning

현재 trajectory layer는 수업에서 다루는 기본 개념을 반영한다.

- line과 arc primitive를 시간 샘플링한다.
- quintic time scaling을 기본으로 사용한다.
- speed, acceleration, jerk limit을 적용한다.
- 그림 정확도와 접촉 안정성을 위해 drawing speed는 느린 편으로 둔다.
- pen-up 구간은 접촉이 풀릴 때 command가 갑자기 튀지 않도록 release logic을 적용한다.

3.5 Controller / JADE

현재 기본 제어는 full torque controller가 아니라, 안정적인 joint-position backend 기반 구조이다.

```
Task-space DLS IK
+ pen-axis orientation task
+ tangent XY correction
+ normal-force admittance
+ JADE execution monitoring
```

JADE는 다음을 기록하고 평가한다.

- planned vs actual XY path
- tracking error
- tip orientation error

- normal contact force
- force offset
- singular value
- condition number
- manipulability
- joint-limit margin

4. 시뮬레이션이 돌아가는 방식

현재 데모 실행 흐름은 다음과 같다.

1. 사용자가 자연어처럼 command를 입력한다.
예: "중앙에 반지름 4cm짜리 원을 그려줘"
2. llm_isaac_bridge가 no-api fallback planner를 실행한다.
실제 OpenAI API를 쓰지 않고, 정해진 규칙으로 DrawingPlan JSON을 만든다.
3. bridge_validator가 plan을 검사한다.
action 순서, board 범위, 속도, pen state, arc/line 연결성을 확인한다.
4. 기존 run_jade_isaac.py가 실행된다.
DrawingPlan JSON을 trajectory로 샘플링하고, board frame을 robot base frame으로 변환한다.
5. pen-tip offset을 적용한다.
목표 pen-tip pose에서 목표 panda_link7 pose를 계산한다.
6. Isaac backend가 Franka state와 Jacobian을 읽는다.
q, qdot, EE pose, Jacobian, contact force를 가져온다.
7. controller가 panda_joint1부터 panda_joint7까지의 target을 만든다.
DLS IK와 force admittance를 사용한다.
8. Isaac Sim이 step을 진행한다.
robot이 table 위 paper에서 펜을 움직인다.
9. 결과가 outputs에 저장된다.
CSV log, summary JSON, planned-vs-actual plot, force plot 등이 생성된다.

5. 발표용 no-api 데모 입력

현재 발표용으로 고정한 입력은 다음 6개이다.

case id	자연어 입력	결과
`circle_r4cm`	`중앙에 반지름 4cm짜리 원을 그려줘`	원
`square_s6cm`	`중앙에 한 변 6cm짜리 사각형을 그려줘`	사각형
`triangle_s6cm`	`중앙에 한 변 6cm짜리 삼각형을 그려줘`	삼각형
`letter_a_8cm`	`중앙에 알파벳 A를 8cm 크기로 그려줘`	알파벳 A
`house_8cm`	`중앙에 8cm 집 모양을 그려줘`	집 모양
`star_8cm`	`중앙에 8cm 별을 그려줘`	별

이 입력들은 실제 LLM reasoning은 아니고, 발표용으로 안정적으로 재현 가능한 rule-based fallback이다.

발표에서는 다음처럼 설명하는 것이 정확하다.

현재 데모에서는 API 비용 없이 재현 가능한 rule-based natural-language fallback을 사용했다.
실제 LLM planner는 같은 DrawingPlan schema로 교체 가능하도록 분리해두었다.

6. 데모 실행 방법

6.1 plan 생성과 validation

```
cd /home/kimchangyeol/IsaacLab/IR  
/home/kimchangyeol/IsaacLab/IR/llm_isaac_bridge/scripts/prepare_demo_showcase.sh
```

생성되는 파일:

```
outputs/demo_showcase/plans/*.json  
outputs/demo_showcase/reports/*_validation.json  
outputs/demo_showcase/manifest.json  
outputs/demo_showcase/isaac_commands.sh
```

6.2 특정 case만 Isaac에서 실행

```
cd /home/kimchangyeol/IsaacLab/IR  
  
/home/kimchangyeol/IsaacLab/IR/llm_isaac_bridge/scripts/prepare_demo_showcase.sh --case  
→ circle_r4cm --execute-isaac
```

circle_r4cm 대신 다음 case id를 넣을 수 있다.

```
square_s6cm  
triangle_s6cm  
letter_a_8cm  
house_8cm  
star_8cm
```

영상 녹화는 각 case를 하나씩 실행하고, Isaac Sim 화면을 직접 화면 녹화하는 방식을 권장한다.

6.3 전체 case 명령 모음

```
bash /home/kimchangyeol/IsaacLab/IR/outputs/demo_showcase/isaac_commands.sh
```

단, 영상 녹화용으로는 전체 연속 실행보다 case 하나씩 실행하는 것이 안정적이다.

7. 현재 상태에서 가능한 것과 불가능한 것

가능한 것

- API 없이 자연어 형태의 입력으로 plan 생성
- DrawingPlan JSON validation
- circle, square, triangle, A, house, star 데모 plan 생성
- Isaac Sim에서 Franka가 pen-tip trajectory를 따라 움직이는 실행
- contact force, tracking error, orientation error, Jacobian metric logging
- 결과 plot과 summary 생성

아직 제한적인 것

- no-api fallback은 실제 LLM이 아니다.
- 자유로운 자연어 명령 전체를 이해하지는 못한다.
- 현재 fallback은 중앙 기준 도형 위주이다.
- 실제 종이에 잉크가 남는 rendering은 아직 없다.
- 제어기 성능은 계속 개선 중이다.
- 실제 로봇 하드웨어 연동은 아직 없다.
- OpenAI API 기반 완전 자동 LLM planner는 API 비용과 환경 분리가 필요하다.

8. 앞으로 추가하면 좋은 고급 기능

8.1 LLM / Planner 쪽

- OpenAI API 또는 local LLM 기반 실제 natural-language planner 연결
- 더 다양한 도형과 복합 그림 지원
- 위치 표현 지원: 왼쪽 위, 오른쪽 아래, 두 개의 도형, 크기 비교 등
- LLM 출력에 대한 stronger schema validation
- plan 수정 요청 기능: “조금 더 작게”, “오른쪽으로 옮겨줘”

8.2 Trajectory / Planning 쪽

- corner에서 자동 감속
- multi-stroke drawing의 stroke order 최적화
- paper boundary와 robot reachability를 함께 보는 pre-check
- velocity/acceleration feedforward 추가
- 실제 handwriting path 또는 SVG path 변환

8.3 Controller / JADE 쪽

- contact force spike 감소
- guarded pen-down 고도화
- normal-force admittance 안정화
- tangent tracking error 보정 개선
- singularity와 joint-limit에 따른 adaptive speed scaling 강화
- optional torque hybrid force-position controller를 고급 모드로 추가
- 더 정교한 null-space posture control

8.4 Isaac / Visualization 쪽

- 자동 camera pose 설정
- 자동 screen recording 또는 Isaac viewport capture
- ink trail rendering
- 결과 plot을 하나의 report로 자동 묶기
- demo case별 success/fail dashboard 생성

8.5 협업 / GitHub 쪽

- IR 프로젝트를 독립 GitHub repository로 공유
- README에 no-api demo와 Isaac 실행 방법 추가
- 팀원이 실행할 수 있는 dependency/setup guide 정리
- 발표용 demo checklist 작성

9. 현재 팀원에게 공유할 핵심 메시지

1. 기존 LLM planner 이후의 robot-control, Isaac scene, bridge, JADE 실행 계층은 현재 별도로 구현되어 있다.
2. 현재는 API 없이도 자연어 형태의 입력을 넣어 Isaac Sim에서 Franka drawing demo를 보여줄 수 있다.
3. 실제 LLM을 붙일 준비는 되어 있지만, API 비용과 환경 분리 문제가 있으므로 발표용은 no-api fallback이 더 안정적이다.
4. 제어기는 아직 최종 완성 상태가 아니며, contact force와 trajectory tracking은 계속 개선할 예정이다.
5. 다음 회의에서는 고급 기능을 어디까지 넣을지 정해야 한다. 특히 실제 LLM, 더 많은 도형, force control 고도화, 자동 녹화/리포트 중 우선순위를 정하면 된다.